U25AI055

DSA Lab-9

PART 1 :

1. **Given two arrays a[] and b[] of size m and n respectively, the task is to determine whether b[] is a subset of a[]. Both arrays are not sorted, and elements are distinct. Input: a[] = [11, 1, 13, 21, 3, 7], b[] = [11, 3, 7, 1] Output: true**

```
#include<iostream>
using namespace std;
int main()
{
  int m, n;
  cout<<"Enter m ";
  cin>>m;
  cout<<"Enter n ";
  cin>>n;
  int a[m], b[n];
  cout<<"Enter A elements ";
  for(int i=0; i<m; i++)
    cin>>a[i];
  cout<<"Enter B elements ";
  for(int i=0; i<n; i++)
    cin>>b[i];
  int j;
  for(j=0; j<n; j++)
  {
    bool flag=false;
    for(int i=0; i<m && !flag; i++)
    {
      if(a[i]==b[j])
        flag=true;
    }
    if(!flag)
      break;
  }
  if(j==n)
    cout<<"B subset of A\n";
  else
    cout<<"B not subset of A\n";
```

```
  return 0;
}
```

Output :

```
Enter m 6
Enter n 4
Enter A elements 11
1
13
21
3
7
Enter B elements 11
3
7
1
B subset of A
```

2. **Given an array arr[], the task is to find the maximum distance between two occurrences of any element. If no element occurs twice, return 0. Input: arr = [1, 1, 2, 2, 2, 1] Output: 5 Explanation: distance for 1 is: 5-0 = 5, distance for 2 is: 4-2 = 2, So max distance is 5.**

```
#include<iostream>
using namespace std;
int main()
{
  int n;
  cout<<"Enter number of elements ";
  cin>>n;
  int arr[n];
  cout<<"Enter elements ";
  for(int i=0; i<n; i++)
    cin>>arr[i];
  int max=0;
  for(int i=0; i<=n-2; i++)
  {
    for(int j=i+1; j<=n-1; j++)
    {
      if(arr[i]==arr[j] && j-i>max)
        max=j-i;
    }
  }
  cout<<"Maximum difference between two occurences of
elements="<<max<<endl;
  return 0;
```

```
}
```

Output :

```
Enter number of elements 6
Enter elements 1
1
2
2
2
1
Maximum difference between two occurences of
 elements=5
```

3. **Given two arrays a[] and b[], find their intersection — the unique elements that appear in both. Ignore duplicates, and the result can be in any order. Input: a[] = [1, 2, 1, 3, 1], b[] = [3, 1, 3, 4, 1] Output: [1, 3] Explanation: 1 and 3 are the only common elements and we need to print only one occurrence of common elements**

```
#include<iostream>
#include<vector>
using namespace std;
void sort_array(int arr[], int n);
int main()
{
  int m, n;
  cout<<"Enter m ";
  cin>>m;
  cout<<"Enter n ";
  cin>>n;
  int a[m], b[n];
  cout<<"Enter 1st array elements ";
  for(int i=0; i<m; i++)
    cin>>a[i];
  cout<<"Enter 2nd array elements ";
  for(int i=0; i<n; i++)
    cin>>b[i];
  sort_array(a, m);
  sort_array(b, n);
  int arr[(m>n) ? m : n], count=0;
  for(int i=0, j=0; i<m && j<n;)
  {
    if(a[i]==b[j])
    {
      arr[count++]=a[i];
      i++;
```

```
        j++;
      }
      else if(a[i]<b[j])
        i++;
      else
        j++;
    }
    for(int i=0; i<count; i++)
      cout<<arr[i]<<"\t";
    return 0;
  }
  void sort_array(int arr[], int n)
  {
    for(int pass=1; pass<=n-1; pass++)
    {
      for(int i=0; i<pass-i; i++)
      {
        if(arr[i]>arr[i+1])
        {
          int temp=arr[i];
          arr[i]=arr[i+1];
          arr[i+1]=temp;
        }
      }
    }
  }
```

Output :

```
Enter m 5
Enter n 5
Enter 1st array elements 1
2
1
3
1
Enter 2nd array elements 3
1
3
4
1
1       3
```

4. **Given an array arr[] of n integers and a target value, find the number of pairs of integers in the array whose sum is equal to target. Input: arr[] = [1, 5, 7, -1, 5], target = 6 Output: 3 Explanation: Pairs with sum 6 are (1, 5), (7, -1) & (1, 5).**

```
#include<iostream>
#include<vector>
```

```
using namespace std;
int find_pairs(int arr[], int n, int target);
int main()
{
   int n, target;
   cout<<"Enter number of elements ";
   cin>>n;
   int arr[n];
   cout<<"Enter elements ";
   for(int i=0; i<n; i++)
      cin>>arr[i];
   cout<<"Enter target ";
   cin>>target;
   cout<<"Number of pairs="<<find_pairs(arr, n, target)<<endl;
   return 0;
}
int find_pairs(int arr[], int n, int target)
{
   int count=0;
   for(int i=0; i<=n-2; i++)
   {
      for(int j=i+1; j<=n-1; j++)
      {
         if(arr[i]+arr[j]==target)
            count++;
      }
   }
   return count;
}
```

Output :

```
Enter number of elements 5
Enter elements 1
5
7
-1
5
Enter target 6
Number of pairs=3
```

PART 2 :

1) **Implementation of Hash Table (Division Method)**

//Implementation of hash table (Division method)

```
#include<iostream>
using namespace std;
struct Node
{
   int data;
   Node *next;
};
int main()
{
   int m, n;
   cout<<"Enter m ";
   cin>>m;
   Node *table[m];
   for(int i=0; i<m; i++)
      table[i]=NULL;
   cout<<"Enter number of keys ";
   cin>>n;
   int keys[n];
   cout<<"Enter keys ";
   for(int i=0; i<n; i++)
      cin>>keys[i];
   for(int i=0; i<n; i++)
   {
      int j=keys[i]%m;
      Node *new_node;
      new_node=new Node;
      new_node->data=keys[i];
      new_node->next=NULL;
      if(table[j]==NULL)
         table[j]=new_node;
      else
      {
         new_node->next=table[j];
         table[j]=new_node;
      }
   }
   for(int i=0; i<m; i++)
   {
      Node *p;
      p=table[i];
      cout<<"Cell="<<i<<"\t";
```

```
    while(p!=NULL)
    {
      cout<<p->data<<"->";
      p=p->next;
    }
    cout<<endl;
  }
  return 0;
}
```

Output :

```
Enter m 10
Enter number of keys 7
Enter keys 50
700
76
85
92
73
101
Cell=0  700->50->
Cell=1  101->
Cell=2  92->
Cell=3  73->
Cell=4
Cell=5  85->
Cell=6  76->
Cell=7
Cell=8
Cell=9
```

2) Multiplication Hash Function

```
//Multiplication hash function
#include<iostream>
#include<math.h>
using namespace std;
struct Node
{
  int data;
  Node *next;
};
int main()
{
  int m, n;
  cout<<"Enter m ";
  cin>>m;
  Node *table[m];
  for(int i=0; i<m; i++)
    table[i]=NULL;
```

```
  cout<<"Enter number of keys ";
  cin>>n;
  int keys[n];
  cout<<"Enter keys ";
  for(int i=0; i<n; i++)
    cin>>keys[i];
  const float A=(sqrt(5)-1)/2;
  for(int i=0; i<n; i++)
  {
    int j=(int)(m*(keys[i]*A-floor(keys[i]*A)));
    Node *new_node;
    new_node=new Node;
    new_node->data=keys[i];
    new_node->next=NULL;
    if(table[j]==NULL)
      table[j]=new_node;
    else
    {
      new_node->next=table[j];
      table[j]=new_node;
    }
  }
  for(int i=0; i<m; i++)
  {
    cout<<"Cell="<<i<<"\t";
    Node *p;
    p=table[i];
    while(p!=NULL)
    {
      cout<<p->data<<"->";
      p=p->next;
    }
    cout<<endl;
  }
  return 0;
}
```

Output :

```
Enter m 10
Enter number of keys 7
Enter keys 50
700
76
85
92
73
101
Cell=0
Cell=1  73->
Cell=2
Cell=3
Cell=4  101->
Cell=5  85->
Cell=6  700->
Cell=7
Cell=8  92->
Cell=9  76->50->
```

3) **Mid-Square and Folding Methods**

```
//Mid-square method
#include<iostream>
#include<math.h>
using namespace std;
struct Node
{
   int data;
   Node *next;
};
int count_digits(int num);
int main()
{
   int n;
   Node *table[100];
   for(int i=0; i<100; i++)
      table[i]=NULL;
   cout<<"Enter n ";
   cin>>n;
   int keys[n];
   cout<<"Enter keys ";
   for(int i=0; i<n; i++)
      cin>>keys[i];
   for(int i=0; i<n; i++)
   {
      int square=keys[i]*keys[i], count=count_digits(square);
      float remove=(count-2)/2;
      int middle=((int)(square/pow(10, remove)))%(int)pow(10, 2);
```

```
        int j=middle%100;
        Node *new_node;
        new_node=new Node;
        new_node->data=keys[i];
        new_node->next=NULL;
        if(table[j]==NULL)
            table[j]=new_node;
        else
        {
            new_node->next=table[j];
            table[j]=new_node;
        }
    }
    for(int i=0; i<100; i++)
    {
        Node *p;
        p=table[i];
        cout<<"Cell="<<i<<"\t";
        while(p!=NULL)
        {
            cout<<p->data<<"->";
            p=p->next;
        }
        cout<<endl;
    }
    return 0;
}
int count_digits(int num)
{
    int count=0;
    while(num>0)
    {
        count++;
        num/=10;
    }
    return count;
}
```

Output :

```
Enter n 1
Enter keys 44
```

```
Cell=94 44->
```

4) **Collision Resolution Using Linear Probing**

```
//Linear probing
#include<iostream>
using namespace std;
int main()
{
  int m, n;
  cout<<"Enter m ";
  cin>>m;
  int table[m];
  for(int i=0; i<m; i++)
    table[i]=-1;
  cout<<"Enter number of keys ";
  cin>>n;
  int keys[n];
  cout<<"Enter keys ";
  for(int i=0; i<n; i++)
    cin>>keys[i];
  for(int j=0; j<n; j++)
  {
    int i=0, k;
    do
    {
      k=(keys[j]+i)%m;
      if(table[k]!=-1)
        i++;
    }while(table[k]!=-1 && i<=5);
    if(table[k]==-1)
      table[k]=keys[j];
  }
  for(int i=0; i<m; i++)
    cout<<"Cell="<<i<<"\t"<<table[i]<<endl;
  return 0;
}
```

Output :

```
Enter m 10
Enter number of keys 9
Enter keys 55
99
61
89
78
74
38
56
84
Cell=0  89
Cell=1  61
Cell=2  38
Cell=3  -1
Cell=4  74
Cell=5  55
Cell=6  56
Cell=7  84
Cell=8  78
Cell=9  99
```

5) **Collision Resolution Using Quadratic Probing**

```
//Quadratic probing
#include<iostream>
using namespace std;
int main()
{
  int m, n;
  cout<<"Enter m ";
  cin>>m;
  int table[m];
  for(int i=0; i<m; i++)
    table[i]=-1;
  cout<<"Enter number of keys ";
  cin>>n;
  int keys[n];
  cout<<"Enter keys ";
  for(int i=0; i<n; i++)
    cin>>keys[i];
  for(int j=0; j<n; j++)
  {
    int i=0, k;
    do
    {
      k=(keys[j]+i*i)%m;
      if(table[k]!=-1)
```

```
            i++;
        }while(table[k]!=-1 && i<=5);
        if(table[k]==-1)
            table[k]=keys[j];
    }
    for(int i=0; i<m; i++)
        cout<<"Cell="<<i<<"\t"<<table[i]<<endl;
    return 0;
}
```

Output :

```
Enter m 10
Enter number of keys 10
Enter keys 55
99
61
89
78
74
38
56
84
69
Cell=0  89
Cell=1  61
Cell=2  38
Cell=3  84
Cell=4  74
Cell=5  55
Cell=6  56
Cell=7  -1
Cell=8  78
Cell=9  99
```

6) Collision Resolution Using Double Hashing

```
//Double hashing
#include<iostream>
using namespace std;
int main()
{
    int m, n;
    cout<<"Enter m ";
    cin>>m;
    int table[m];
    for(int i=0; i<m; i++)
        table[i]=-1;
    cout<<"Enter number of keys ";
    cin>>n;
    int keys[n];
```

```
    cout<<"Enter keys ";
    for(int i=0; i<n; i++)
        cin>>keys[i];
    for(int j=0; j<n; j++)
    {
        int i=0, k;
        do
        {
            k=(keys[j]%m+i*((1+keys[j])%(m-2)))%m;
            if(table[k]!=-1)
                i++;
        }while(table[k]!=-1 && i<=5);
        if(table[k]==-1)
            table[k]=keys[j];
    }
    for(int i=0; i<m; i++)
        cout<<"Cell="<<i<<"\t"<<table[i]<<endl;
    return 0;
}
```

Output :

```
Enter m 10
Enter number of keys 9
Enter keys 55
99
61
89
78
74
38
56
84
Cell=0  -1
Cell=1  61
Cell=2  38
Cell=3  89
Cell=4  74
Cell=5  55
Cell=6  56
Cell=7  -1
Cell=8  78
Cell=9  99
```

7) **Collision Resolution Using Chaining**

```
//Chaining
#include<iostream>
using namespace std;
struct Node
```

```
{
   int data;
   Node *next;
};
int main()
{
   int m, n;
   cout<<"Enter m ";
   cin>>m;
   Node *table[m];
   for(int i=0; i<m; i++)
      table[i]=NULL;
   cout<<"Enter number of keys ";
   cin>>n;
   int keys[n];
   cout<<"Enter keys ";
   for(int i=0; i<n; i++)
      cin>>keys[i];
   for(int i=0; i<n; i++)
   {
      int j=keys[i]%m;
      Node *new_node;
      new_node=new Node;
      new_node->data=keys[i];
      new_node->next=NULL;
      if(table[j]==NULL)
         table[j]=new_node;
      else
      {
         new_node->next=table[j];
         table[j]=new_node;
      }
   }
   for(int i=0; i<m; i++)
   {
      Node *p;
      p=table[i];
      cout<<"Cell="<<i<<"\t";
      while(p!=NULL)
      {
         cout<<p->data<<"->";
```

```
        p=p->next;
      }
      cout<<endl;
    }
    return 0;
}
```

Output :

```
Enter m 10
Enter number of keys 7
Enter keys 50
700
76
85
92
73
101
Cell=0  700->50->
Cell=1  101->
Cell=2  92->
Cell=3  73->
Cell=4
Cell=5  85->
Cell=6  76->
Cell=7
Cell=8
Cell=9
```